

RFID Gen2x Development Documentation

V2. 16

Table of contents

1 Gen2x new feature related interfaces	3
1.1 setExtProfile 【Setting the Gen2x Profile 】	3

Enable Gen2X Function Description:

Gen2X is Impinj's high-performance reading mode optimized for M700 and M800 series tags.

Depending on your needs, you can choose from two reading strategies::

Configuration

Profile Number	Read Tag range
type	
Dedicated	
Mode	4123、4124、4141、4146、4148、4185 Read only M700, M800 series tags
Compatible	
Mode	5123、5124、5141、5146、5148、5185 Read M700, M800 series tags + ordinary tags

Note: The Gen2X functionality can only be enabled after setting the Profile to one of the 12 options listed above.

1 Gen2x new feature related interfaces

1.1 setExtProfile 【Setting the Gen2x Profile 】

definition	public int setExtProfile(int ExtProfile);		
illustrate	Set the Profile for Gen2X. It can also be set through the interface "setProfile(RfidProfile rfidProfile);", and the effect is the same, although the parameter type is different. Note: The Profile will only change if it is actively set. If a specific Profile for Gen2X has been set, when returning to the normal mode and using the standard inventory method (such as: startInventory()), please check according to your business requirements whether to revert to the normal supported Profile (not the Gen2X-specific Profile).		
parameter	name	type	Remark
	btOutputPower	int	scope: 1, 3, 5, 7

			11, 12, 13, 15 102, 103, 104, 120, 123, 124 125, 126, 141, 146, 147, 148 185, 202, 203, 205, 222, 223 224, 225, 226, 241, 244, 285 302, 323, 324, 325, 326, 342 343, 344, 345, 382 4123, 4124, 4141, 4146, 4148, 4185 5123, 5124, 5141, 5146, 5148, 5185 For more details, see: Appendix 2. Profile details supported by Gen2x
return (int)	Success: 0 ; Failure: other ;		
Reference Code	none		

1.2 getExtProfile [Query Gen2x Profile]

definition	public int getExtProfile() ;		
illustrate	Query the current profile.		
parameter	name	type	Remark
return (int)	Returns the output power (failed to obtain if less than 0)		
Reference Code	none		

1.3 SetShortRangeFlag 【 Set short range mode】

definition	public int SetShortRangeFlag(int epcNum, String strEPC, String strPassword, int MaskMemInt, int btWordAdd, byte MaskLength, String strMaskData, int srBit, int srValue)		
illustrate	This command is used to set the short-range mode of the Tag.		
parameter	name	type	Remark
	epcNum	int	The EPC number length of the tag. When Enum = 255, the mask mode is used.
	strEPC	String	Enum words, the EPC number of the tag.
	strPassword	String	4 bytes, tag access password.
	MaskMemInt	int	Mask Bank. 1 – EPC Bank; 2 – TID Bank; 3 – User Bank.
	btWordAdd	int	The starting bit address of the mask, ranging from 0 to 16383. For example, if the EPC is 1111 in E28011113333, the starting position is 12 (because there are 2 words of data before E280 in the EPC)
	MaskLength	int	The bit length of the mask. For example, if the data is "1111", the length here is 4.
	strMaskData	String	Mask data. Character length is an even number
	srBit	int	Short -range mode storage location, Impinj700/800 tag SRBit = 4; Impinj MR6-X tag SRBit = 1;
	srValue	int	0-Short-range mode is turned off ; 1- Turn on short -range mode.
return (void)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.		
Reference Code	none		

1.4 getShortRangeFlag 【Read short range mode】

definition	public int getShortRangeFlag(int epcNum, String strEPC, String strPassword, int MaskMemInt, int btWordAdd, byte MaskLength, String strMaskData, int srBit)
-------------------	--

illustrate	This command is used to read whether the tag is set to Short-range mode.		
parameter	name	type	Remark
	epcNum	int	The EPC number length of the tag. When Enum = 255, the mask mode is used.
	strEPC	String	ENum words, the EPC number of the tag.
	strPassword	String	4 bytes, tag access password.
	MaskMemInt	int	Mask area. 1 – EPC Bank; 2 – TID Bank; 3 – User Bank.
	btWordAdd	int	bit address of the mask , ranging from 0 to 16383. For example, if the EPC is 1111 in E28011113333, the starting position is 12 (because there are 2 words of data before E280 in the EPC)
	MaskLength	int	bit length of the mask . For example, if the data is "1111", the length here is 4.
	strMaskData	String	Mask data. Character length is an even number
	srBit	int	The bit position stored in short-range mode , Impinj700/800 tag SRBit = 4; Impinj MR6-X tag SRBit = 1;
return (void)	0-turn off Short-range mode; 1- Turn on Short-range mode. Other: Indicates failure. Please refer to the return value error code table.		
Reference Code	none		

1.5 marginRead [Verify the reliability of data writing]

definition	public int marginRead (int epcNum, String strEPC, int MemInt, int address,byte matchLength, String strMatchdata, String strPassword,int MaskMemInt,int btWordAdd,int MaskLength,String strMaskData)		
illustrate	This command is used to verify the reliability of data writing.		
parameter	name	type	Remark

	epcNum	int	1 byte, the length of the tag's EPC number. When Enum = 255, the mask mode is used.
	strEPC	String	ENum words, the EPC number of the tag.
	MemInt	int	1 byte, Bank. 10 – Reserved area; 1 – EPC Bank; 2 – TID Bank; 3 – User Bank.
	address	int	The starting bit of the Bank (the length of the string)
	matchLength	int	Match length (string length in bits)
	matchdata	String	Data to be matched
	strPassword	String	4 bytes, tag access password.
	MaskMemInt	int	Mask area. 1 – EPC Bank; 2 – TID Bank; 3 – User Bank.
	btWordAdd	int	The starting bit address of the mask, ranging from 0 to 16383. For example, if the EPC is 1111 in E28011113333, the starting position is 12 (because there are 2 words of data before E280 in the EPC) The bit length of the mask. For example, if the data is "1111", the length here is 4.
	MaskLength	int	The bit length of the mask. For example, if the data is "1111", the length here is 4. Mask data. Character length is an even number
	strMaskData	String	Mask data. Character length is an even number
return (void)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.		
Reference Code	none		

1.6 authenticate Impinj tags

definition	public AuthRes authenticate(String epc, String password)		
illustrate	This command is used to authenticate the Impinj tag.		
parameter	name	type	Remark
	epc	String	EPC of the Tag
	password	String	4 bytes, tag access password.
return (AuthRes)	AuthRes Description: int code; //0: indicates success. Not equal to 0: indicates failure. Please		

	refer to the return value error code table. String random; //Random number String response; //Returned tag authentication data.
Reference Code	none

1.7 . setPowerBoost 【 Set PowerBoost command 】

definition	public int setPowerBoost (int enableFlag)		
illustrate	This command is used to set the Power boost flag and is limited to the Ex10 series.		
parameter	name	type	Remark
	enableFlag	int	0-off; 1-on
return (int)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.		
Reference Code	none		

1.8 getPowerBoost 【 Read PowerBoost command 】

definition	public int getPowerBoost ()		
illustrate	This command is used to read the TagFocus tag and is limited to the Ex10 series.		
parameter	name	type	Remark
return (int)	0-off; 1-on ; other: indicates failure, please check the return value error code table.		
Reference Code	none		

1.9 setTagFocus [Set TagFocus command]

definition	public int setFocus (int mode)			
illustrate	This command is used to set the TagFocus tag and is only applicable to the Ex10 series.			
parameter	name	type	direction	Remark
	mode	int	[in]	0-off; 1-on
return (int)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.			
Reference Code	none			

1.10 getFocus [Read TagFocus command]

definition	public int getFocus ()			
illustrate	This command is used to read the TagFocus tag and is limited to the Ex10 series.			
parameter	name	type	direction	Remark
return (int)	0-off; 1-on; other: indicates failure, please check the return value error code table.			
Reference Code	none			

1.11 setImpinjScanParam 【Set Impinj Scanc command parameters】

definition	public int setImpinjScanParam (int opt,int N,int Code,int CR,int Protection,int ID, int CopyTo)
------------	---

illustrate	This command is used to set Impinj scan command parameters and is limited to the EX10 series. After setting, you can call the startRead method to perform inventory scanning.		
parameter	name	type	Remark
	opt	int	0-Save after power failure; 1-Do not save after power failure
	N	int	Default 0x0F;
	Code	int	0-Rfu;1-Antipodal;2-CCOneHalf;3-CCThreeQuarters.
	CR	int	0-ID32;1-ID16;2-StoredCRC;3-RN16.
	Protection	int	0-NoProtection;1-Parity;2-CRC5;3-CRC5Plus.
	ID	int	0-NoAckResponse; 1-TMNPlusTSN; 2-Part EPC; 3-Full.
	CopyTo	int	0-All tags; 1-S=1 tags only.
return (int)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.		
Reference Code	none		

1.12 getImpinjScanParam 【Read Impinj Scanc command parameters 】

definition	<pre>public int getImpinjScanParam() int[] aar = RFIDSDKManager.getInstance().getRfidManager().getImpinjScanParam();</pre>		
illustrate	This command is used to read Impinj scan command parameters and is limited to the EX10 series.		
parameter	name	type	Remark
	N --> aar[0]	int	Default 0x0F;
	Code--> aar[1]	int	0-Rfu;1-Antipodal;2-CCOneHalf;3-CCThreeQuarters.
	CR--> aar[2]	int	0-ID32;1-ID16;2-StoredCRC;3-RN16.
	Protection--> aar[3]	int	0-NoProtection;1-Parity;2-CRC5;3-CRC5Plus.
	ID--> aar[4]	int	0-NoAckResponse; 1-TMNPlusTSN; 2-Part EPC; 3-Full.
	CopyTo--> aar[5]	int	0-All tags; 1-S=1 tags only.

return (int)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table. If 0 is returned, the corresponding parameter value is obtained by reading the passed parameter.
Reference Code	none

1.13 setInventoryMatchData 【Multi-function mask】

definition	public int setInventoryMatchData(int MatchType, List<String> epcs)		
illustrate	This command is used to set multiple masks at the same time and is only applicable to the Ex10 series.		
parameter	name	type	Remark
	MatchType	int	0-match; 1-no match
	epcs	List<String>	Mask list, up to 5 groups. When it is empty, it means clearing the mask list;
return (int)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.		
Reference Code	none		

1.14 setTagQueting [enable or disable TagQuieting]

definition	public int setTagQueting(int Opt,int enableFlag)		
illustrate	This command is used to set whether to enable the internally implemented TagQuieting algorithm tag. Customers can also use the universal multi-function mask to implement it themselves. This command is limited to the Ex10 series.		
parameter	name	type	Remark
	Opt	int	0-Save after power failure; 1-Do not save after power failure
	mode	boolean	0-off; 1-on

return (int)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.
Reference Code	none

1.15 getTagQueting [Get the status of the TagQuieting setting]

definition	public int getTagQueting ()		
illustrate	This command is used to read whether the internal TagQuieting algorithm is enabled. Customers can also use the universal multi-function mask to implement it themselves. This command is only for the Ex10 series.		
parameter	name	type	Remark
return (int)	0 - Close; 1 - Open Other: Indicates failure, please check the return value error code table.		
Reference Code	none		

1.16 ProtectedMode [Set privacy mode for tags]

definition	public int protectedMode(String EPCStr, int enable, String Password);		
illustrate	Before setting the privacy mode, you must change the access password, and the modified access password must be consistent with the one used when setting the privacy mode.		
parameter	name	type	Remark
	EPCStr	String	Selected EPC number
	enable	int	0: Disable privacy mode 1: Enable privacy mode
	Password	String	Set the access password in privacy mode. The password must be the same as the modified access password.
return (int)	Success: 0; Failure: Other.		
Reference Code			

1.16 setReaderProtectedMode [Set reader privacy mode]

definition	<code>public int setReaderProtectedMode(int Opt,int enableFlag,String strPassword);</code>		
illustrate	This command is used to set the privacy mode of the reader and is limited to the Ex10 series.		
parameter	name	type	Remark
	Opt	int	0-Save after power failure; 1-Do not save after power failure
	enableFlag	int	0-off; 1-on
	strPassword	String	4 bytes, access password used internally in the reader's privacy mode
return (int)	0: Success. Not equal to 0: Failure. Please refer to the return value error code table.		
Reference Code	none		

1.17 getReaderProtectedMode [Read the reader privacy mode]

definition	<code>public String[] getReaderProtectedMode();</code>		
illustrate	This command is used to read whether the reader is set to privacy mode. It is only applicable to the Ex10 series.		
parameter	name	type	Remark
return (int)	Get successful: return the collection String[] aar; String enable = aar[0]; // 0-disable; 1-enable String password = aar[1]; // 4 bytes, access password used internally in the reader's privacy mode Get success: return null object		
Reference Code	none		

Appendix 1 : **Common Error Codes**

Error Code	describe
0	Execution successful.
1	No electronic tag found.
2	Insufficient power. The tag has insufficient power to perform storage write operations.
3	Memory limit exceeded or unsupported PC value, the specified memory location does not exist or is marked The signature does not support PC value.
4	Memory Lock: A memory location is permanently locked and cannot be written to.
5	The access password is incorrect.
9	Destroy password error.
10	The destroy password cannot be all 0s.
11	Electronic Tag do not support this command.

12	For this command, the access password cannot be 0.
13	The electronic tag has been read-protected and cannot be read-protected again.
14	The electronic tag is not read-protected and does not need to be unlocked.
15	Non-specific error. The tag does not support specific error codes.
16	The write failed because the byte space is locked.
17	Cannot be locked.
18	Already locked, cannot be locked again.
19	Parameter saving failed, but the set value is valid before the read/write module is powered off.
20	Cannot be adjusted.
248	Antenna detection error
249	An error occurred while executing the command.
250	There is an electronic tag, but the communication is poor and it cannot be operated.
251	No electronic tags can be operated.
252	The electronic tag returns an error code.
253	The command length is incorrect.
254	Illegal command.
255	The parameters are incorrect.
48	Communication error.

Appendix 2: Profile details supported by Gen2x

"1: PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 2",

"3: PR_ASK, pie: 2.0, tari_us: 20.0, lf_khz: 320, M: 2",

"5: PR_ASK, pie: 2.0, tari_us: 20.0, lf_khz: 320, M: 4",

"7: PR_ASK, pie: 2.0, tari_us: 20.0, lf_khz: 250, M: 4",

"11: PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 1",
"12: PR_ASK, pie: 2.0, tari_us: 15.0, lf_khz: 320, M: 2",
"13: PR_ASK, pie: 2.0, tari_us: 20.0, lf_khz: 160, M: 8",
"15: PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 4",
"102:PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 1",
"103:DSB, pie: 1.5, tari_us: 6.25,lf_khz: 640, M: 1",
"104:DSB, pie: 1.5, tari_us: 6.25,lf_khz: 320, M: 1",
"120:DSB, pie: 1.5, tari_us: 6.25,lf_khz: 640, M: 2",
"123:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 320, M: 2",
"124:PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 2",
"125:PR_ASK, pie: 2.0, tari_us: 15.0,lf_khz: 320, M: 2",
"126:PR_ASK, pie: 1.5, tari_us: 12.5,lf_khz: 320, M: 2",
"141:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 320, M: 4",
"146:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 250, M: 4",
"147:PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 4",
"148:PR_ASK, pie: 1.5, tari_us: 7.5, lf_khz: 640, M: 4",
"185:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 160, M: 8",
"202:PR_ASK, pie: 2.0, tari_us: 15.0,lf_khz: 426, M: 1",
"203:PR_ASK, pie: 1.5, tari_us: 12.5, lf_khz: 426, M: 1",
"205:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 50, M: 1",
"222:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 320, M: 2",
"223:PR_ASK, pie: 2.0, tari_us: 15.0,lf_khz: 320, M: 2",
"224:PR_ASK, pie: 1.5, tari_us: 12.5,lf_khz: 320, M: 2",
"225:PR_ASK, pie: 2.0, tari_us: 15.0,lf_khz: 426, M: 2",
"226:PR_ASK, pie: 1.5, tari_us: 12.5,lf_khz: 426, M: 2",
"241:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 320, M: 4",

"244:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 250, M: 4",
"285:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 160, M: 8",
"302:PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 1",
"323:PR_ASK, pie: 2.0, tari_us: 7.5, lf_khz: 640, M: 2",
"324:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 320, M: 2",
"325:PR_ASK, pie: 2.0, tari_us: 15.0,lf_khz: 320, M: 2",
"326:PR_ASK, pie: 1.5, tari_us: 12.5,lf_khz: 320, M: 2",
"342:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 320, M: 4",
"343:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 250, M: 4",
"344:PR_ASK, pie: 2.0, tari_us: 7.5,lf_khz: 640, M: 4",
"345:PR_ASK, pie: 1.5, tari_us: 7.5 ,lf_khz: 640, M: 4",
"382:PR_ASK, pie: 2.0, tari_us: 20.0,lf_khz: 160, M: 8",
"4123:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 320, PSK2",
"4124:PR_ASK,pie: 2.0, tari_us: 7.5, lf_khz: 640, BPSK2"
"4141:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 320, BPSK4"
"4146:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 250, BPSK4"
"4148:PR_ASK,pie: 1.5, tari_us: 7.5, lf_khz: 640, BPSK4"
"4185:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 160, BPSK8"
"5123:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 320, M+B2",
"5124:PR_ASK,pie: 2.0, tari_us: 7.5, lf_khz: 640, M+B2",
"5141:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 320, M+B4",
"5146:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 250, M+B4",
"5148:PR_ASK,pie: 1.5, tari_us: 7.5, lf_khz: 640, M+B4",
"5185:PR_ASK,pie: 2.0, tari_us: 20.0,lf_khz: 160, M+B8"

Appendix 3: RSSI parameter comparison table

RSSI parameters	Signal strength	RSSI parameters	Signal strength
98 (0x62)	-31dBm	64 (0x40)	-66dBm
97 (0x61)	-32dBm	63 (0x3F)	-67dBm
96 (0x60)	-33dBm	62 (0x3E)	-68dBm
95(0x5F)	-34dBm	61(0x3D)	-69dBm
94 (0x5E)	-35dBm	60(0x3C)	-70dBm
93(0x5D)	-36dBm	59 (0x3B)	-71dBm
92 (0x5C)	-37dBm	58 (0x3A)	-72dBm
91(0x5B)	-38dBm	57 (0x39)	-73dBm
90(0x5A)	-39dBm	56 (0x38)	-74dBm
89 (0x59)	-41dBm	55 (0x37)	-75dBm
88 (0x58)	-42dBm	54 (0x36)	-76dBm
87 (0x57)	-43dBm	53 (0x35)	-77dBm
86 (0x56)	-44dBm	52 (0x34)	-78dBm
85 (0x55)	-45dBm	51 (0x33)	-79dBm
84 (0x54)	-46dBm	50 (0x32)	-80dBm
83 (0x53)	-47dBm	49 (0x31)	-81dBm
82 (0x52)	-48dBm	48 (0x30)	-82dBm
81 (0x51)	-49dBm	47 (0x2F)	-83dBm
80 (0x50)	-50dBm	46 (0x2E)	-84dBm
79 (0x4F)	-51dBm	45(0x2D)	-85dBm
78 (0x4E)	-52dBm	44 (0x2C)	-86dBm
77 (0x4D)	-53dBm	43 (0x2B)	-87dBm

76 (0x4C)	-54dBm	42 (0x2A)	-88dBm
75 (0x4B)	-55dBm	41 (0x29)	-89dBm
74 (0x4A)	-56dBm	40(0x28)	-90dBm
73 (0x49)	-57dBm	39(0x27)	-91dBm
72 (0x48)	-58dBm	38 (0x26)	-92dBm
71 (0x47)	-59dBm	37 (0x25)	-93dBm
70 (0x46)	-60dBm	36 (0x24)	-94dBm
69 (0x45)	-61dBm	35(0x23)	-95dBm
68 (0x44)	-62dBm	34 (0x22)	-96dBm
67 (0x43)	-63dBm	33 (0x21)	-97dBm
66 (0x42)	-64dBm	32 (0x20)	-98dBm
65 (0x41)	-55dBm	31 (0x1F)	-99dBm